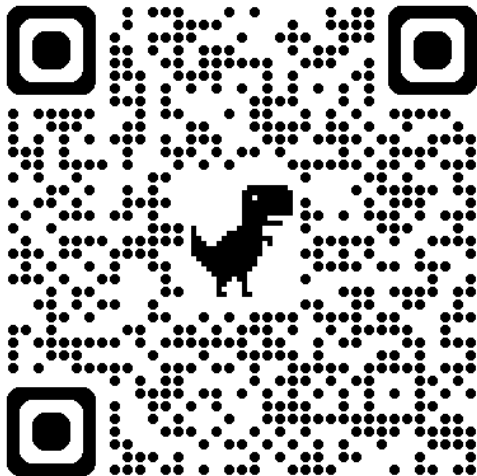


Stack and Queue

Tai, Wei Hsuan

week 6



1. Recap Problems

2. Stack

3. Queue

4. Practice Problems

Recap Problems

Stack

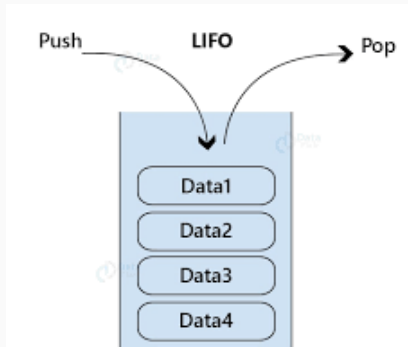
What is stack?

In English, stack means "a pile of things arranged one on top of another(Cambridge Dictionary)". In C++, stack is a container adapter that gives the programmer the functionality of a stack data structure.

How a stack works?

Stack is a Last In First Out (LIFO) data structure. The last element added to the stack will be the first one to be removed.

In other words, you can only access the top element of the stack(i.e., the most recently added element). You can push elements onto the stack, remove elements from the stack, and check the top element of the stack.



Basic usage of stack

```
1      //create a stack of TYPE named s:  
2      stack<TYPE> s;  
3      //add an element to the top of the stack:  
4      s.push(ELEMENT);  
5      //remove the top element of the stack:  
6      s.pop();  
7      //access the top element of the stack:  
8      s.top();  
9      //access the number of elements in the stack:  
10     s.size();  
11     //check if the stack is empty:  
12     s.empty();
```

If you want to access the top element of the stack or remove the top element of the stack, you should first check if the stack is empty to avoid runtime errors.

Here are some common operations of stack

```
1      stack<int> s; //create a stack of integers
2      s.push(1); //push 1 onto the stack
3      s.push(2); //push 2 onto the stack
4      cout<<s.top(); //print the top element of the stack
5      s.pop(); //remove the top element of the stack
```

Queue

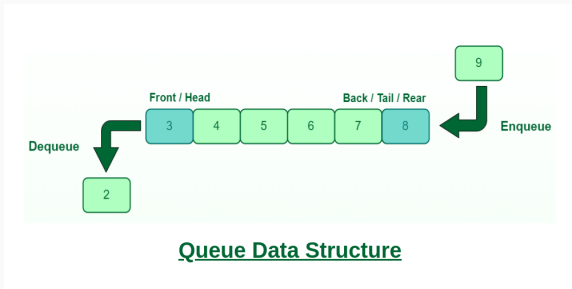
What is queue?

In English, queue means "a line of people, usually standing or in cars, waiting for something, or a lot of people who want something(Cambridge Dictionary)". In C++, queue is a container that gives the programmer the functionality of a queue data structure.

How a queue works?

Queue is a First In First Out (FIFO) data structure. The first element added to the queue will be the first one to be removed.

In other words, you can only access the front and rear elements of the queue. You can push elements to the rear of the queue, remove elements from the front of the queue, and check the front and rear elements of the queue.

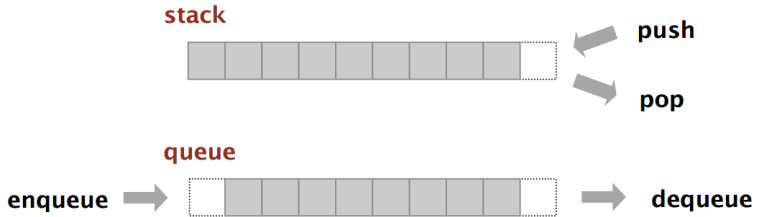


Basic usage of queue

```
1      //create a queue of TYPE named q:
2      queue<TYPE> q;
3      //add an element to the rear of the queue:
4      q.push(ELEMENT);
5      //remove the front element of the queue:
6      q.pop();
7      //access the front element of the queue:
8      q.front();
9      //access the rear element of the queue:
10     q.back();
11     //access the number of elements in the queue:
12     q.size();
13     //check if the queue is empty:
14     q.empty();
```

If you want to access or remove any elements in the queue, you should first check if the queue is empty to avoid runtime errors.

Stack vs Queue



Practice Problems

Problems

Please refer to **week 6 Stack & Queue** on zerojudge.

- Basic Stack Problems: i213, a565
- Basic Queue Problems: e447
- Advanced Stack Problems: c700, j607, r395
- Advanced Queue Problems: d718

I'll mainly focus on the basic problems, the advanced problems are optional.